Intro to Numpy

NumPy কী?

NumPy (Numerical Python) হলো পাইথনের একটি লাইব্রেরি, যা বৈজ্ঞানিক ও গাণিতিক হিসাব করার জন্য ব্যবহার করা হয়।
এটি মূলত array নিয়ে কাজ করে।

মূল বৈশিষ্ট্য

- **ndarray (N-dimensional array)** → NumPy এর প্রধান অবজেক্ট।
- এটি এক বা একাধিক মাত্রার (1D, 2D, 3D...) ডেটা সংরক্ষণ করতে পারে।
- একই ডেটা টাইপ → NumPy অ্যারেতে সব উপাদানের ডেটা টাইপ একই হয় । যেমন সব int বা সব float।
- দ্রুত গাণিতিক অপারেশন → বড় ডেটাসেটের ওপর যোগ, গুণ, ভাগ ইত্যাদি খুব দ্রুত করা যায়।
- কম কোডে বেশি কাজ → সাধারণ পাইথন লুপ ব্যবহার না করেও অনেক অপারেশন করা যায়।

কেন **NumPy** ব্যবহার করবেন

- পাইথনের list এর চেয়ে অনেক দ্রুত কাজ করে।
- বড় ডেটা সহজে হ্যান্ডেল করা যায়।
- ম্যাথ, স্ট্যাটিসটিকস, মেশিন লার্নিং, ডেটা অ্যানালাইসিসে এইটি অপরিহার্য।

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)  # output: [5 7 9]
```

ndarray and its attributes

ndarray কী?

ndarray (N-dimensional array) হলো NumPy এর সবচেয়ে গুরুত্বপূর্ণ অবজেক্ট।
এটি এমন একটি স্ট্রাকচার যেখানে একই ডেটা টাইপের অনেক ডেটা একসাথে রাখা যায়।
একে পাইথনের list এর মতো মনে হলেও, এটি অনেক দ্রুত এবং বড় ডেটার জন্য কার্যকরী।

Array Type

```
import numpy as np

# 1D array
arr1 = np.array([1, 2, 3, 4, 5])

# 2D array
arr2 = np.array([[1, 2, 3],
                 [4, 5, 6]])

# 3D array
arr3 = np.array([
    [[1, 2, 3], [4, 5, 6]],
    [[1, 2, 3], [4, 5, 6]],
    [[1, 2, 3], [4, 5, 6]]
])
```

- 1D: এক লাইন ডেটা → [1,2,3,4,5]
- 2D: সারি ও কলাম আকারে → টেবিলের মতো
- 3D: একাধিক 2D টেবিল

Array Attributes

ndim

এটি বলে দেয় অ্যারেটি কয় dimensional এর।

- print(arr1.ndim) # 1
- print(arr2.ndim) # 2
- print(arr3.ndim) # 3

shape

এটি বলে দেয় প্রতিটি dimension এ কয়টি উপাদান আছে ,যেমন সারি ও কলাম সংখ্যা।

```
print(arr1.shape)  # (5,)
print(arr2.shape)  # (2, 3)
print(arr3.shape)  # (3, 2, 3)
```

size

এটি বলে দেয় অ্যারেতে মোট কয়টি উপাদান আছে।

```
print(arr3.size)  # 18
```

ndarray Data Type

ndarray Data Type

NumPy এর **ndarray** এ সব element একই type এর হতে হয়।
যেমন → সবগুলো সংখ্যা হলে int বা float হবে, কিন্তু ভিন্ন টাইপ হলে NumPy সেটিকে এক টাইপে রূপান্তর করে নেয়।

dtype দেখা

```
import numpy as np

arr = np.array([1, 2, 3])
print(arr.dtype)    # int64

arr = np.array([1.5, 2.4, 3.2])
print(arr.dtype)    # float64
```

এখানে প্রথম অ্যারে শুধুই পূর্ণসংখ্যা, তাই int64
দ্বিতীয় অ্যারেতে দশমিক আছে, তাই float64

Upcasting →এক টাইপে রূপান্তর করা

```
arr = np.array([1, 2, 3.2])
print(arr.dtype)    # float64
```

এখানে int ও float একসাথে আছে, তাই সবগুলো **float** হয়ে গেছে।

সংখ্যার সাথে string থাকলে

```
arr = np.array([1, 2, 3.2, 'hello'])
print(arr.dtype)    # <U32
```

NumPy সংখ্যাকে string-এ রূপান্তর করেছে।
এইখানে U মানে Unicode string।

মিশ্র ডেটা (int, bool, float, string)

```
arr = np.array([1, True, 3.25, 'hello'])
print(arr.dtype)    # <U32
```

যেহেতু string আছে, তাই সব উপাদান string হয়ে গেছে।

ডেটা টাইপ সেট করে দেয়া

```
arr = np.array([1, 2, 300], dtype=np.int16)
```

```
print(arr.dtype)   # int16
print(arr)         # [  1   2 300]
```

.astype() দিয়ে পুরানো অ্যারের কপি তৈরি হয় নতুন টাইপে।

ভুল টাইপ দিলে এরর আসবে

```
arr = np.array([1, 2, 3.2, 'hello'], dtype=np.float64)
```

এখানে string আছে, কিন্তু dtype দেওয়া হয়েছে float64 → Error: ValueError: could not convert string to float: 'hello'

ndarray creation from Existing data

ndarray Creation from Existing Data

NumPy এর ndarray শুধু নতুনভাবে নয়, বরং আগে থেকেই থাকা ডেটা →যেমন list, tuple, set, dictionary থেকেও তৈরি করা যায়।
এভাবে তৈরি করার সময় চাইলে ডেটা টাইপ নির্ধারণ করা যায়।

List থেকে ndarray তৈরি

```
1import numpy as np
2
3lst = [10, 20, 30, 40, 40.5]
4arr = np.array(lst, dtype=np.int32)
```

এখানে **list** থেকে array বানানো হয়েছে।
dtype=np.int32 দিয়ে ডেটা টাইপ নির্ধারণ করা হয়েছে।

```
1mixed_lst = [10, True, 'Hello']
2arr = np.array(mixed_lst)
```

ভিন্ন টাইপ থাকলে (সংখ্যা + string), তাহলে NumPy সবকিছু string এ upcast করে নেয়।

Nested List থেকে **ndarray** তৈরি

```
1matrix = [[1, 2, 3],
           [4, 5, 6]]
3
4arr = np.array(matrix)
```

আউটপুট:

```
[[1 2 3]
 [4 5 6]]
```

টাইপ → <class 'numpy.ndarray'>
ডেটা টাইপ → int64
shape → (2, 3)
Dimension → 2
এটি একটি 2D অ্যারে, অর্থাৎ সারি ও কলাম আকারে সাজানো ডেটা।

Tuple থেকে ndarray তৈরি

```
tpl = (10, 20, 30)
arr = np.array(tpl, dtype=np.int32)
```

এখানে **tuple** থেকে অ্যারে তৈরি করা হয়েছে।

```
arr = arr.astype(np.int16)
```

.astype() ব্যবহার করে ডেটা টাইপ পরিবর্তন করা হয়েছে।
shape → (3,)
ndim → 1
dtype → int16

Set থেকে ndarray তৈরি

```
st = {1, 2, 3}
arr = np.array(list(st))
arr = arr.astype(np.int16)
```

যেহেতু **set** unordered, তাই list() দিয়ে রূপান্তর করে তারপর NumPy অ্যারে বানানো হয়েছে।
shape → (3,)
ndim → 1
dtype → int16

Dictionary থেকে ndarray তৈরি

```
dc = {'a': 10, 'b': 20, 'c': 30}
```

Keys থেকে ndarray

```
arr = np.array(list(dc.keys()))
```

Output: ['a' 'b' 'c']
dtype → <U1 (Unicode string)
shape → (3,)

ndim → 1

Values থেকে ndarray

```
arr = np.array(list(dc.values()))

arr = arr.astype(np.int16)
```

Output: [10 20 30]

dtype → int16

shape → (3,)

ndim → 1

Items (key-value pair) থেকে ndarray

```
arr = np.array(list(dc.items()))
```

Output:

```
[['a' '10']
['b' '20']
['c' '30']]
```

dtype → <U21

shape → (3, 2)

ndim → 2 এটি একটি **2D** অ্যারে, যেখানে প্রতিটি সারিতে key ও value আছে।

ndarray creation from scratch

Creating ndarray from Scratch

NumPy দিয়ে আমরা একদম শুরু থেকে নতুন অ্যারে তৈরি করতে পারি।

এর জন্য কিছু built-in ফাংশন আছে যেমন → zeros(), ones(), empty(), full() ইত্যাদি।

np.zeros() → 0 দিয়ে অ্যারে তৈরি

```
import numpy as np

arr = np.zeros((2,3), dtype=np.int8)

print(arr)
```

এখানে (2,3) মানে ২ সারি ও ৩ কলামের 2D অ্যারে তৈরি হবে।

সব উপাদান হবে 0।

output:

```
[[0 0 0]
[0 0 0]]
```

এরপর

```
arr2 = np.zeros_like(arr)

print(arr2)
```

এটি arr-এর মতো একই shape ও dtype নিয়ে নতুন অ্যারে বানায়,
কিন্তু সব উপাদান হবে 0।

output:

```
[[0 0 0]

[0 0 0]]
```

np.ones() → 1 দিয়ে অ্যারে তৈরি

```
arr = np.ones((3,4,3), dtype=np.int8)

print(arr)
```

এখানে 3D অ্যারে তৈরি হবে যেখানে সব মান 1।

output:

```
[[[1 1 1]
  [1 1 1]
  [1 1 1]
  [1 1 1]]

 [[1 1 1]
  [1 1 1]
  [1 1 1]
  [1 1 1]]

 [[1 1 1]
  [1 1 1]
  [1 1 1]
  [1 1 1]]]
```

এরপর

```
arr2 = np.ones_like(arr)
```

```
print(arr2)
```

এটি arr এর shape অনুযায়ী একে দিয়ে পুরো নতুন অ্যারে তৈরি করে।

output:

```
[[[1 1 1]
  [1 1 1]
  [1 1 1]
  [1 1 1]]

[[1 1 1]
  [1 1 1]
  [1 1 1]
  [1 1 1]]

[[1 1 1]
  [1 1 1]
  [1 1 1]
  [1 1 1]]]
```

np.empty() → খালি (random value দিয়ে) অ্যারে তৈরি

```
arr = np.empty((4,3), dtype=np.int8)
print(arr)
```

এখানে NumPy শুধু মেমোরি বরাদ্দ করে, মান দেয় না।
ফলে এলোমেলো (garbage) মান দেখা যেতে পারে।

output:

```
[[ 1  0  0]
 [ 0  2  0]
 [ 0  0 44]
 [ 1  0  0]]
```

এরপর

```
arr2 = np.empty_like(arr)
```

```
2print(arr2)
```

এটি arr এর মতো একই shape ও dtype নিয়ে খালি অ্যারে তৈরি করে।
ভেতরের মান এলোমেলো হতে পারে
shape → (3, 2, 3)
output:

```
1[[ 1  0  0]
2[ 0  2  0]
3[ 0  0 44]
4[ 1  0  0]]
```

np.full() → নির্দিষ্ট মান দিয়ে অ্যারে তৈরি

```
1arr = np.full((4,3), np.inf)
2print(arr)
```

এখানে সব উপাদান হবে **infinity (∞)** মানে দিয়ে ভর্তি।
output:

```
1[[inf inf inf]
2[inf inf inf]
3[inf inf inf]
4[inf inf inf]]
```

এরপর

```
1arr2 = np.full_like(arr, np.inf, dtype=np.float64)
2print(arr2)
```

এটি arr এর মতো shape নিয়ে float টাইপের ইনফিনিটি মানে ভর্তি অ্যারে তৈরি করে।
output:

```
1[[inf inf inf]
2[inf inf inf]
3[inf inf inf]
4[inf inf inf]]
```

ndarray generation with random values

Array Creation with Random Values

NumPy তে আমরা np.random মডিউল ব্যবহার করে র‍্যান্ডম মানসহ অ্যারে তৈরি করতে পারি।
এগুলো সাধারণত ডেটা সিমুলেশন, মেশিন লার্নিং, বা পরীক্ষামূলক ডেটা তৈরিতে ব্যবহৃত হয়।

np.random.rand() → 0 থেকে 1 এর মধ্যে মান

```
import numpy as np

arr = np.random.rand(2, 3)
```

এখানে 2 সারি ও 3 কলামের অ্যারে তৈরি হবে,
যেখানে প্রতিটি মান হবে 0 থেকে 1 এর মধ্যে দশমিক সংখ্যা।

output:

```
[[0.32 0.84 0.57]
[0.15 0.99 0.42]]
```

dtype → float64
shape → (2, 3)
ndim → 2
size → 6

np.random.randint() → নির্দিষ্ট রেঞ্জের পূর্ণসংখ্যা

```
arr = np.random.randint(1, 10, (2, 3))
```

এটি ১ থেকে ৯ এর মধ্যে র‍্যান্ডম integer মান তৈরি করে।
(2,3) মানে ২ সারি × ৩ কলামের অ্যারে।

output:

```
[[3 9 1]
[7 5 2]]
```

dtype → int64
shape → (2, 3)
ndim → 2
size → 6

np.random.uniform() → নির্দিষ্ট রেঞ্জের দশমিক মান

```
arr = np.random.uniform(50, 100, (3, 3, 4))
```

এটি 50 থেকে 100 এর মধ্যে র‍্যান্ডম float মান তৈরি করে।
(3,3,4) মানে 3D অ্যারে।

output:

```
[[[63.2 89.4 70.1 55.3]
  [79.0 92.6 68.4 54.7]
  [59.8 75.2 99.9 61.3]]

 [[81.4 97.1 53.6 71.5]
  [90.2 69.4 85.7 66.3]
  [51.7 80.9 95.1 99.0]]

 [[61.3 53.1 70.8 93.4]
  [88.2 78.1 50.9 83.6]
  [97.8 74.4 56.7 59.9]]]
```

dtype → float64
shape → (3, 3, 4)
ndim → 3
size → 36

ndarray creation from range

Array Creation with Range Functions

NumPy তে কিছু বিশেষ ফাংশন আছে যা নির্দিষ্ট রেঞ্জ বা ধাপে ধাপে মান দিয়ে অ্যারে তৈরি করতে ব্যবহৃত হয়।
এগুলো হলো np.arange(), np.linspace(), ও np.logspace()।

np.arange() → নির্দিষ্ট ধাপে মান তৈরি

এটি পাইথনের range() ফাংশনের মতো, কিন্তু NumPy অ্যারে রিটার্ন করে।

```
import numpy as np

arr = np.arange(1, 10, 1)
```

1 = শুরু মান
10 = শেষ মান (ধরা হয় না)
1 = ধাপ (step size)
output:

```
[1 2 3 4 5 6 7 8 9]
```

এরপর

```
mat = arr.reshape(3,3)
```

reshape() ব্যবহার করে এটিকে 3×3 ম্যাট্রিক্সে রূপান্তর করা হয়েছে।

ouput:

```
[[1 2 3]
[4 5 6]
[7 8 9]]
```

dtype → int64

shape → (9,) বা (3,3)

ndim → 1 বা 2

size → 9

np.linspace() → নির্দিষ্ট সংখ্যক সমান দূরত্বের মান তৈরি

এটি **start** থেকে **end** পর্যন্ত মান তৈরি করে এবং মোট কয়টি মান লাগবে তা নির্ধারণ করা যায়।

```
arr = np.linspace(0, 4, 6)
```

0 = শুরু মান

4 = শেষ মান

6 = মোট কয়টি মান লাগবে

output:

```
[0.  0.8 1.6 2.4 3.2 4. ]
```

এখানে 0 থেকে 4 পর্যন্ত 6টি সমান ব্যবধানের float মান তৈরি হয়েছে।

dtype → float64

shape → (6,)

ndim → 1

size → 6

np.logspace() → লগারিদমিক স্কেলে মান তৈরি

এটি লগ স্কেলে (base অনুযায়ী) মান তৈরি করে।

সাধারণত বিজ্ঞান ও ডেটা স্কেলিং-এ ব্যবহৃত হয়।

```
arr = np.logspace(0, 4, 6, base=2)
```

0 → 4 : লগ মানের রেঞ্জ

base=2 : লগের ভিত্তি

6 : মোট মানের সংখ্যা

output:

```
[ 1.          2.5198421   6.34960421 16.          40.3174736 102.4
]
```

এখানে 2^0 থেকে 2^4 পর্যন্ত 6টি মান তৈরি হয়েছে।

creating diagonal and identity matrix

Creating Matrix for Linear Algebra

NumPy তে রৈখিক বীজগণিত (Linear Algebra) এর জন্য বিভিন্ন ধরণের বিশেষ ম্যাট্রিক্স সহজেই তৈরি করা যায়।
সবচেয়ে সাধারণ হলো → Diagonal Matrix ও Identity Matrix।

Diagonal Matrix

np.diag() ফাংশন ব্যবহার করে ম্যাট্রিক্সের মূল কর্ণে (diagonal) মান বসানো যায়।

```
import numpy as np

diagonal_matrix = np.diag([1, 2, 3, 4])
print(diagonal_matrix)
print(diagonal_matrix.shape)
```

output:

```
[[1 0 0 0]
[0 2 0 0]
[0 0 3 0]
[0 0 0 4]]
(4, 4)
```

কর্ণে (diagonal) মান বসে — [1, 2, 3, 4]
বাকি সব জায়গায় 0
shape → (4, 4)
dtype → int64
এই ধরণের ম্যাট্রিক্স অনেক লিনিয়ার অ্যালজেব্রা অপারেশনে ব্যবহৃত হয় (যেমন determinant, eigenvalue ইত্যাদি)।

Identity Matrix

Identity Matrix হলো এমন একটি স্কয়ার ম্যাট্রিক্স যেখানে কর্ণে (diagonal) সব মান 1 এবং বাকিগুলো 0।

```
identity_mat = np.eye(4)
```

output:

```
[[1. 0. 0. 0.]
```

```
[0. 1. 0. 0.]
[0. 0. 1. 0.]
[0. 0. 0. 1.]]
```

shape → (4, 4)

কর্ণে ১, বাকিগুলো ০

dtype → float64

np.eye() → Custom Identity Matrix

np.eye() দিয়ে চাইলে সারি ও কলামের সংখ্যা আলাদা করে দেওয়া যায়।

```
mat = np.eye(3, 4)
print(mat)
```

output:

```
[[1. 0. 0. 0.]
[0. 1. 0. 0.]
[0. 0. 1. 0.]]
```

এখানে ৩ সারি, ৪ কলাম → shape (3, 4)

কর্ণে ১ বসে

np.eye() with k parameter

k প্যারামিটার ব্যবহার করে কর্ণের অবস্থান উপরে বা নিচে সরানো যায়।

```
mat = np.eye(3, 4, 1)
print(mat)
```

output:

```
[[0. 1. 0. 0.]
[0. 0. 1. 0.]
[0. 0. 0. 1.]]
```

k = 1 → মূল কর্ণের এক ধাপ উপরে

k = -1 → দিলে নিচের দিকে সরে যায়

ndarray Indexing and Slicing

Indexing and Slicing

NumPy array থেকে নির্দিষ্ট মান, সারি বা কলাম বেছে নেওয়ার জন্য আমরা Indexing ও Slicing ব্যবহার করি।

এগুলো লিস্টের মতোই কাজ করে, তবে অনেক বেশি powerful ও দ্রুত।

Indexing

ইনডেক্স ব্যবহার করে অ্যারের নির্দিষ্ট element বেছে নেওয়া হয়।

ইনডেক্স শুরু হয় **0** থেকে।

মাল্টিডাইমেনশনাল অ্যারেতে (যেমন 2D বা 3D) একাধিক ইনডেক্স লাগে।

```
import numpy as np

arr1 = np.array([1, 2, 3, 4, 5])
arr2 = np.array([[1, 2, 3],
                 [4, 5, 6]])
```

একটি মান পরিবর্তন করা:

```
arr2[0][2] = 100
print(arr2)
```

ouptut:

```
[[  1   2 100]
 [  4   5   6]]
```

arr2[0][2] মানে → প্রথম সারির (row 0) তৃতীয় উপাদান (column 2)

নতুন মান 100 বসানো হয়েছে।

Slicing

অ্যারের একটি অংশ (subarray) বের করার জন্য স্লাইসিং ব্যবহার করা হয়।

Syntax:

```
arr[start:end:step]
```

start → কোথা থেকে শুরু

end → কোথায় থামবে

step → কয় ধাপ পর পর মান নেবে

1D Array Slicing

```
arr1 = np.array([1, 2, 3, 4, 5])
arr1_mod = arr1[1:4].copy()
arr1[1] = 2
arr1_mod[2] = 200

print(arr1_mod)
```

```
7print(arr1)
```

ouput:

```
1[  2   3 200]

2[1 2 3 4 5]
```

arr1[1:4] মানে index 1 থেকে 3 পর্যন্ত অংশ নেওয়া।
.copy() দিলে আলাদা কপি তৈরি হয় (মূল অ্যারেতে পরিবর্তন হলে এতে প্রভাব পড়ে না)।

2D Array Slicing

২ ডাইমেনশনের অ্যারেতে, স্লাইসিং করা হয় এইভাবে

```
1arr[row_start:row_end , col_start:col_end]
```

একটি সম্পূর্ণ সারি নেওয়া

```
1arr2 = np.array([[1, 2, 3],
2                 [4, 5, 6]])
3
4row_0 = arr2[0:1, ]   # প্রথম সারি
5row_1 = arr2[1:, ]    # দ্বিতীয় সারি
```

output:

```
1[[1 2 3]]

2[[4 5 6]]
```

একটি কলাম নেওয়া

```
1arr2 = np.array([[1, 2, 3],
2                 [4, 5, 6]])
3
4col_0 = arr2[:, 0:1]  # প্রথম কলাম
5col_1 = arr2[:, 1:2]  # দ্বিতীয় কলাম
6
7print(col_0)
8print(col_1)
```

output:

```
1[[1]
2[4]]
```

```
3[[2]

4[5]]
```

: মানে → সব সারি নাও
0:1 মানে → শুধুমাত্র প্রথম কলাম
একটি অংশ নেওয়া

```
1arr2 = np.array([[1, 2, 3],
2                 [4, 5, 6]])
3portion = arr2[:, 0:2]

4print(portion)
```

output:

```
1[[  1   2]

2[  4   5]]
```

এটি মূলত ম্যাট্রিক্সের একটি অংশ।

Ndarray Copy Advanced Indexing and Iteration

Advanced Indexing

NumPy তে ইনডেক্সিং শুধু সাধারণ সংখ্যা দিয়েই নয় → বরং index এর list, boolean শর্ত, এমনকি লুপ দিয়েও করা যায়। এগুলোকে একসাথে বলা হয় Advanced Indexing।

Integer Indexing

ইনডেক্সের list দিয়ে নির্দিষ্ট মানগুলো বেছে নেওয়া যায়।

```
1import numpy as np

2

3lst = np.array([10, 20, 30, 40])
4values = lst[[0, 3, 1]]

5

6print(values)
```

output:

```
1[10 40 20]
```

এখানে [0,3,1] মানে → প্রথম, চতুর্থ, ও দ্বিতীয় element বেছে নেওয়া হয়েছে।

2D Integer Indexing

2D অ্যারের সারি ও কলাম নির্দিষ্ট করে একাধিক মান একসাথে নেওয়া যায়।

```
arr2 = np.array([[1, 2, 3],
                 [4, 5, 6]])

print(arr2[[0, 1], [1, 2]])
```

output:

```
[2 6]
```

প্রথম ইনডেক্স [0,1] → সারি
দ্বিতীয় ইনডেক্স [1,2] → কলাম
অর্থাৎ → (0,1) ও (1,2) পজিশনের মান নেওয়া হয়েছে।

Boolean Indexing

কোনো শর্ত অনুযায়ী মান বেছে নেওয়া যায়।

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

print(arr[arr > 1])
```

output:

```
[2 3 4 5 6]
```

এখানে arr > 1 একটি boolean অ্যারে তৈরি করে।
যেখানে শর্ত True, সেই মানগুলো বেছে নেওয়া হয়।

Boolean শর্ত দিয়ে মান পরিবর্তন করা

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

arr[arr > 2] = 0

print(arr)
```

output:

```
[[1 2 0]
[0 0 0]]
```

যেসব মান > 2, সেগুলোকে 0 এ পরিবর্তন করা হয়েছে।

Iterating Over Arrays

np.nditer() ব্যবহার করে multi-dimensional অ্যারে থেকেও একে একে মান নেওয়া যায়।

```
arr3 = np.array([
    [[1, 2, 3],
     [4, 5, 6]],
    [[7, 8, 9],
     [10, 11, 12]]
])

for i in np.nditer(arr3):
    print(i)
```

output:

```
1
2
3
4
5
6
7
8
9
10
11
12
```

np.nditer() দিয়ে যেকোনো dimension এর অ্যারে থেকে এক এক করে সব মান পাওয়া যায়।
এটি nested loop ছাড়াই সব মান ঘুরে দেখতে সাহায্য করে।

ndarray Manipulation

এই সেকশনে আমরা গভীরভাবে শিখব কিভাবে **NumPy ndarray (n-dimensional array)** manipulate করতে হয়। Python list এর তুলনায় ndarray অনেক বেশি efficient, কারণ এটি C-based contiguous memory layout ব্যবহার করে এবং বড় data নিয়ে কাজ করার জন্য অসাধারণ দ্রুত।

What is ndarray Manipulation?

ndarray Manipulation বলতে বোঝায় array এর structure **update**, **reshape**, **merge**, **split**, বা **delete** করার কাজ। Data science এ এটি ব্যবহৃত হয় dataset prepare করার জন্য, যেমন feature reshape, merge বা split করা।

Creating and Reshaping Arrays

NumPy তে array **create** করতে `np.array()` ব্যবহার করা হয়।

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])
print(arr.shape)  # (6,)

# reshape
reshaped = arr.reshape(2, 3)
print(reshaped)
```

Output:

```
[[1 2 3]
[4 5 6]]
```

এখানে আমরা 1-D array কে 2-D তে **transform** করেছি। অর্থাৎ 6টি element কে 2 row এবং 3 column এর matrix এ সাজানো হয়েছে।

Note: Reshaping শুধুমাত্র তখনই কাজ করবে যখন total element সংখ্যা একই থাকে।

Reshape vs Resize

Function	Behavior	Returns
reshape()	নতুন shape সহ নতুন array **create** করে	New copy
resize()	মূল array এর shape **update** করে	In-place

```
arr = np.arange(6)
arr2 = arr.reshape(3, 2)
print(arr2)

arr.resize(2, 3)
print(arr)
```

Flattening Arrays

একটি multi-dimensional array কে 1-D তে **transform** করতে আমরা **flatten()** বা **ravel()** ব্যবহার করি।

```
arr = np.array([[1, 2], [3, 4]])
flat1 = arr.flatten()
flat2 = arr.ravel()
print("Flatten result:", flat1)
print("Ravel result:", flat2)
```

> `flatten()` সবসময় নতুন copy **create** করে, অর্থাৎ মূল array এর সাথে এর কোনো সম্পর্ক থাকে না।
> `ravel()` সাধারণত view **return** করে, তাই এটি মূল array এর data reference share করে এবং memory-efficient।

```
arr = np.array([[10, 20], [30, 40]])
r = arr.ravel()
r[0] = 999
print("Modified ravel array:", r)
print("Original array also changes:", arr)
```

Output:

```
Modified ravel array: [999  20  30  40]
```

```
2Original array also changes: [[999  20]

3[ 30  40]]
```

Note: যখন তুমি মূল array কে পরিবর্তন না করে শুধুমাত্র copy প্রয়োজন মনে করছ, তখন flatten() ব্যবহার করা নিরাপদ। কিন্তু যখন memory efficiency দরকার এবং original array পরিবর্তন করা acceptable, তখন ravel() সেরা পছন্দ।

Adding or Removing Dimensions

1-D array কে 2-D বা 3-D তে **expand** করা যায় **np.newaxis** বা **expand_dims()** দিয়ে।

```
1arr = np.array([1, 2, 3])
2print(arr.shape)  # (3,)

3

4expanded = np.expand_dims(arr, axis=0)

5print(expanded.shape)  # (1, 3)
```

এছাড়াও, **squeeze()** ফাংশন দিয়ে অতিরিক্ত dimension **delete** করা যায় —

```
1arr = np.array([[[1, 2, 3]]])
2print(arr.shape)  # (1, 1, 3)
3arr = np.squeeze(arr)

4print(arr.shape)  # (3,)
```

Stacking Arrays (Combining)

একাধিক array কে একত্রে **combine** করতে আমরা **stack(), hstack(), vstack()** ব্যবহার করি।

```
1a = np.array([1, 2, 3])
2b = np.array([4, 5, 6])

3

4print(np.stack((a, b)))   # 2-D vertical stack
5print(np.hstack((a, b)))  # horizontal join

6print(np.vstack((a, b)))  # vertical join
```

hstack() — side by side join করে।
vstack() — উপর-নিচে stack করে।
stack() — নতুন dimension **create** করে merge করে।

Splitting Arrays

Array কে একাধিক ভাগে **split** করা যায় `split()` দিয়ে।

```
arr = np.arange(10)
print(np.split(arr, 2))
# Output: [array([0,1,2,3,4]), array([5,6,7,8,9])]
```

তুমি চাইলে unequal অংশেও **split** করতে পারো —

```
np.split(arr, [3, 7])
# Output: [array([0,1,2]), array([3,4,5,6]), array([7,8,9])]
```

Inserting and Deleting Elements

যদিও ndarray immutable, NumPy নতুন copy **create** করে element **insert** বা **delete** করতে দেয়।

```
arr = np.array([1, 2, 3, 4])
arr = np.insert(arr, 2, 99)
print(arr)  # [1 2 99 3 4]

arr = np.delete(arr, 1)
print(arr)  # [1 99 3 4]
```

Note: এই operations গুলো খুব বড় array তে ধীরগতির হতে পারে কারণ নতুন copy create হয়। Memory efficiency র জন্য বড় array তে preallocation strategy ব্যবহার করা উত্তম।

Extended Manipulation Topics

Transpose and Axis Manipulation

Array এর axis **swap** বা **transpose** করার জন্য:

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.T)
# Output:
# [[1 4]
#  [2 5]
#  [3 6]]
```

Axis পরিবর্তন করা যায় `np.transpose(arr, axes=(1,0))` দিয়ে। এটি বিশেষভাবে ব্যবহার হয় image processing ও matrix operations এ।

Changing Data Type (dtype Conversion)

Array এর data type **convert** করা যায় `astype()` দিয়ে।

```
arr = np.array([1.2, 3.5, 4.9])

print(arr.astype(int))  # [1 3 4]
```

এটি useful যখন numeric data কে integer বা float এ পরিবর্তন করতে হয়।

Copy vs View

Array এর slicing বা reshaping করার পর copy না view তৈরি হয়েছে তা বোঝা খুব গুরুত্বপূর্ণ।

```
a = np.array([1, 2, 3])
b = a.view()
b[0] = 99

print(a)  # [99  2  3] — change reflected in original
```

copy(): completely new memory allocation তৈরি করে

view(): same data reference share করে

Concatenation

Stack এর মতোই কিন্তু বেশি flexible। `np.concatenate()` multiple array **merge** করতে পারে:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.concatenate((a, b)))  # [1 2 3 4 5 6]
```

এটি dimension অনুযায়ী horizontally বা vertically concatenate করতে পারে।

Summary Table

Task	Function	Description
Shape পরিবর্তন	reshape(), resize()	shape **update** করা
Flatten array	flatten(), ravel()	multi-D কে 1-D তে **transform** করা
Dimension বাড়ানো/কমানো	expand_dims(), squeeze()	axis **add** বা **delete** করা
Merge array	stack(), hstack(), vstack(), concatenate()	array **combine** করা
Split array	split()	array **split** করা
Insert/Delete	insert(), delete()	element **add** বা **delete** করা
Transpose	T, transpose()	axis **swap** করা
Type Conversion	astype()	data type **convert** করা
Copy/View	copy(), view()	memory reference control

Real-world Example

Machine Learning dataset prepare করার সময় reshape প্রায়ই দরকার হয় —

```
X = np.arange(12)
X = X.reshape(-1, 3)  # -1 মানে NumPy নিজে row সংখ্যা নির্ধারণ করবে
print(X)
```

Output:

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

এভাবে আমরা সহজেই feature matrix **create** করতে পারি, যেখানে প্রতিটি row হলো একেকটি sample এবং column গুলো হলো features।

এখন তুমি ndarray manipulation এর প্রায় সব core concept জেনে গেছ — reshape, transpose, dimension control, stacking, splitting, inserting, deleting, concatenation, এবং dtype conversion পর্যন্ত। এগুলো Python data analysis ও ML preprocessing এর বেসিক building blocks!

ndarray Arithmetic Operators and Mathematical Functions

এই পর্যায়ে আমরা শিখব কিভাবে NumPy এর ndarray এর ওপর বিভিন্ন **Arithmetic Operators** এবং **Mathematical Functions** apply করা যায়। এটি numerical computation, data analysis, এবং machine learning preprocessing এ অত্যন্ত গুরুত্বপূর্ণ।

Arithmetic Operations on ndarrays

NumPy এর বড় সুবিধা হলো element-wise arithmetic operation — অর্থাৎ, array এর প্রতিটি element এর ওপর আলাদা করে operation apply হয়।

```
import numpy as np

arr1 = np.array([10, 20, 30, 40])
arr2 = np.array([1, 2, 3, 4])

print(arr1 + arr2)  # addition
print(arr1 - arr2)  # subtraction
print(arr1 * arr2)  # multiplication
print(arr1 / arr2)  # division
```

Output:

```
[11 22 33 44]
[ 9 18 27 36]
[10 40 90 160]
[10. 10. 10. 10.]
```

Note: NumPy automatically broadcasts array sizes if they are compatible (see next section on broadcasting).

Scalar Operations

Array এর সাথে scalar মান ব্যবহার করলে সেটি প্রত্যেক element এর ওপর apply হয়।

```
arr = np.array([5, 10, 15])
print(arr + 5)  # every element increased by 5
print(arr * 2)  # every element multiplied by 2
```

Output:

```
[10 15 20]
[10 20 30]
```

Comparison Operators

Comparison বা relational operators ও element-wise কাজ করে —

```
arr = np.array([1, 2, 3, 4])
print(arr > 2)
print(arr == 3)
```

Output:

```
[False False  True  True]
[False False  True False]
```

Tip: এই ধরনের boolean array প্রায়ই data filtering এ apply হয়।

In-place Operations

NumPy অনেক ক্ষেত্রে in-place operation সাপোর্ট করে, অর্থাৎ নতুন array **create** না করে মূল array **update** হয়।

```
arr = np.array([1, 2, 3])
arr += 5  # in-place addition
print(arr)
```

Output:

```
[6 7 8]
```

Mathematical Functions

NumPy তে অনেক built-in mathematical function আছে যা fast এবং efficient কারণ এগুলো C-level implementation এ চলে।

Basic Functions

```
arr = np.array([1, 2, 3, 4, 5])
print(np.sum(arr))      # 15
print(np.mean(arr))     # 3.0
print(np.std(arr))      # 1.4142
print(np.min(arr))      # 1
print(np.max(arr))      # 5
```

Trigonometric Functions

```
angles = np.array([0, np.pi/2, np.pi])
print(np.sin(angles))
print(np.cos(angles))
print(np.tan(angles))
```

Exponential and Logarithmic Functions

```
arr = np.array([1, 2, 3])
print(np.exp(arr))     # e^x
print(np.log(arr))     # natural log
print(np.log10(arr))   # base-10 log
```

Rounding and Power Functions

```
arr = np.array([1.2, 2.6, 3.9])
print(np.round(arr))  # round to nearest
print(np.floor(arr))  # round down
print(np.ceil(arr))   # round up

powers = np.array([2, 3, 4])
print(np.power(powers, 2))  # square each element
```

Summary Table

Operation	Function	Description
Addition/Subtraction	+, -, np.add(), np.subtract()	element-wise add or subtract apply
Multiplication/Division	*, /, np.multiply(), np.divide()	element-wise multiply or divide apply
Power	**, np.power()	raise elements to power apply
Exponential	np.exp()	apply e^x computation
Logarithm	np.log(), np.log10()	apply natural/base-10 log
Trigonometric	np.sin(), np.cos(), np.tan()	apply sine, cosine, tangent functions
Rounding	np.round(), np.floor(), np.ceil()	apply rounding operations
Aggregation	np.sum(), np.mean(), np.std(), np.min(), np.max()	apply aggregate statistics

Real-world Example

Data normalization এ mathematical function apply করা খুবই সাধারণ —

```
arr = np.array([10, 20, 30, 40, 50])
normalized = (arr - np.min(arr)) / (np.max(arr) - np.min(arr))
print(normalized)
```

Output:

```
[0.   0.25 0.5  0.75 1.  ]
```

এভাবে আমরা সহজেই numeric data কে 0-1 রেঞ্জে scale করতে পারি, যা machine learning model এর জন্য আদর্শ ইনপুট ফরম্যাট।

NumPy Broadcasting

এই পর্যায়ে আমরা শিখব **NumPy Broadcasting** সম্পর্কে — এটি এমন একটি শক্তিশালী feature যা বিভিন্ন shape এর arrays এর মধ্যে arithmetic operations apply করতে দেয়, কোনো explicit looping বা reshaping ছাড়াই। Broadcasting এর ফলে NumPy খুব দ্রুত এবং memory-efficient ভাবে কাজ করতে পারে।

What is Broadcasting?

Broadcasting এমন একটি প্রক্রিয়া যেখানে NumPy স্বয়ংক্রিয়ভাবে ছোট array কে বড় array এর সাথে match করার জন্য **expand** করে, যাতে element-wise operation সম্ভব হয়।

উদাহরণ:

```
import numpy as np

arr1 = np.array([1, 2, 3])
arr2 = 5

print(arr1 + arr2)
```

Output:

```
[6 7 8]
```

এখানে scalar মান 5 স্বয়ংক্রিয়ভাবে `[5, 5, 5]` আকারে broadcast হয়েছে।

Broadcasting Rules

NumPy broadcasting এর দুটি array এর shape match করার জন্য নিচের নিয়মগুলো অনুসরণ করে —

> **Shape comparison right to left** হয়।
> দুটি dimension এর মধ্যে যদি তারা সমান হয়, broadcasting apply হয়।
> যদি কোনো dimension এর মান 1 হয়, তাহলে সেটি অন্যটির মানে **expand** হয়।
> অন্য কোনো ক্ষেত্রে broadcasting সম্ভব নয় এবং NumPy `ValueError` দেয়।

উদাহরণ:

```
A = np.array([[1, 2, 3], [4, 5, 6]])  # shape (2, 3)
B = np.array([10, 20, 30])            # shape (3,)

print(A + B)
```

Output:

```
[[11 22 33]
[14 25 36]]
```

এখানে `B` কে `[10, 20, 30]` থেকে `(1, 3)` তে expand করা হয়েছে, তারপর `A` এর `(2, 3)` এর সাথে match হয়েছে।

Broadcasting with Higher Dimensions

NumPy multi-dimensional broadcasting সাপোর্ট করে।

```
A = np.array([[[1], [2], [3]]])    # shape (1, 3, 1)
B = np.array([[10, 20, 30]])       # shape (1, 1, 3)
```

```
C = A + B
print(C.shape)
print(C)
```

Output:

```
(1, 3, 3)
[[[11 21 31]
  [12 22 32]
  [13 23 33]]]
```

এখানে NumPy প্রতিটি dimension অনুযায়ী smaller array কে **expand** করে compatible করেছে।

Useful Broadcasting Functions

NumPy তে broadcasting সম্পর্কিত কিছু built-in function আছে যা broadcasting manually control করতে সাহায্য করে।

1. `np.newaxis`

`np.newaxis` ব্যবহার করে array এর dimension বাড়ানো যায় যাতে broadcasting সম্ভব হয়।

```
arr = np.array([1, 2, 3])
print(arr.shape)  # (3,)

arr_col = arr[:, np.newaxis]
print(arr_col.shape)  # (3, 1)
```

এভাবে আমরা array কে column vector বা row vector আকারে পরিবর্তন করে broadcasting সহজ করি।

2. `np.expand_dims()`

`np.expand_dims()` function dimension বাড়ানোর আরেকটি উপায়। এটি নতুন axis যোগ করে shape পরিবর্তন করে।

Syntax:

```
np.expand_dims(array, axis)
```

Example:

```
arr = np.array([1, 2, 3])
new_arr = np.expand_dims(arr, axis=0)
print(new_arr.shape)  # (1, 3)
```

এটি বিশেষভাবে useful যখন তুমি broadcasting এর জন্য নির্দিষ্ট axis যোগ করতে চাও।

3. `np.broadcast_to()`

`np.broadcast_to()` একটি array কে নির্দিষ্ট shape এ **broadcast** করে, কোনো data copy ছাড়াই।

Syntax:

```
np.broadcast_to(array, shape)
```

Example:

```
arr = np.array([1, 2, 3])
result = np.broadcast_to(arr, (3, 3))
print(result)
```

Output:

```
[[1 2 3]
 [1 2 3]
 [1 2 3]]
```

এটি memory-efficient কারণ array duplicate হয় না, শুধু view expand হয়।

4. `np.tile()`

`np.tile()` function array এর repetition তৈরি করে। broadcasting এর মতো এটি automatic নয়, বরং explicit replication করে।

Syntax:

```
np.tile(array, reps)
```

Example:

```
arr = np.array([1, 2, 3])
tiled = np.tile(arr, (2, 3))
print(tiled)
```

Output:

```
[[1 2 3 1 2 3 1 2 3]
 [1 2 3 1 2 3 1 2 3]]
```

Broadcasting virtual expansion করে, কিন্তু np.tile() আসলেই data replicate করে, তাই memory বেশি লাগে।

When Broadcasting Fails

যদি দুইটি array এর shape একে অপরের সাথে অসামঞ্জস্যপূর্ণ হয়, broadcasting কাজ করবে না।

```
A = np.array([[1, 2, 3], [4, 5, 6]])  # shape (2, 3)
B = np.array([[1, 2]])                # shape (1, 2)
```

```
3A + B
```

এক্ষেত্রে NumPy `ValueError: operands could not be broadcast together` দেখাবে কারণ second dimension (3 vs 2) সমান নয়।

Practical Examples of Broadcasting

Example 1: Data Normalization

```
1X = np.array([[10, 20, 30], [40, 50, 60]])
2mean = np.mean(X, axis=0)
3normalized = X - mean  # broadcast mean row to each row of X
4print(normalized)
```

Output:

```
1[[-15. -15. -15.]
2[ 15.  15.  15.]]
```

Example 2: Scaling with Vector

```
1X = np.array([[1, 2, 3], [4, 5, 6]])
2scales = np.array([1, 10, 100])
3scaled = X * scales
4print(scaled)
```

Output:

```
1[[  1  20 300]
2[  4  50 600]]
```

এখানে প্রতিটি column কে আলাদা ভাবে scale করা হয়েছে।

Summary Table

Key Advantages of Broadcasting

> Memory efficiency: ছোট array duplicate না করেই ব্যবহার করা যায়।
> Performance improvement: looping এর দরকার নেই, সবকিছু vectorized ভাবে চলে।
> Simplicity: একই কোড বিভিন্ন shape এর data তে apply করা যায়।

এই পর্যায়ে তুমি Broadcasting এর core concept, rules, helper functions (`np.newaxis`, `np.expand_dims()`, `np.broadcast_to()`, `np.tile()`), এবং practical ব্যবহার বুঝে গেছ — এখন তুমি সহজেই বিভিন্ন shape এর array নিয়ে arithmetic operations apply করতে পারবে NumPy তে।

NumPy Logical Functions

এই পর্যায়ে আমরা শিখব **NumPy Logical Functions** সম্পর্কে — এগুলো এমন ফাংশন যা boolean বা logical operations perform করতে ব্যবহৃত হয়। Data filtering, conditional computation, এবং decision-making এর জন্য এই ফাংশনগুলো অত্যন্ত গুরুত্বপূর্ণ।

Introduction to Logical Operations

NumPy তে সাধারণ তুলনা operators যেমন `>`, `<`, `==`, `!=` ইত্যাদি element-wise comparison করে এবং boolean array return করে।

```
import numpy as np

arr = np.array([10, 20, 30, 40])
print(arr > 25)
```

Output:

```
[False False  True  True]
```

এই ধরনের boolean array পরে filtering বা indexing এ ব্যবহার করা যায়।

```
filtered = arr[arr > 25]
print(filtered)
```

Output:

```
[30 40]
```

Core Logical Functions

NumPy কিছু built-in logical functions দেয় যা একাধিক condition combine করতে ব্যবহৃত হয়।

1. `np.logical_and()`

একাধিক condition একসাথে true কিনা যাচাই করে।

```
arr = np.array([10, 20, 30, 40])
result = np.logical_and(arr > 10, arr < 35)
print(result)
print(arr[result])
```

Output:

```
[False  True  True False]
[20 30]
```

দুটি শর্তই true হলে শুধুমাত্র সেই elements নির্বাচিত হয়।

2. `np.logical_or()`

যেকোনো একটি condition true হলে true return করে।

```
arr = np.array([10, 20, 30, 40])
result = np.logical_or(arr < 15, arr > 35)
print(result)
print(arr[result])
```

Output:

```
[ True False False  True]
[10 40]
```

3. `np.logical_not()`

Boolean array কে invert করে।

```
arr = np.array([10, 20, 30])
condition = arr > 15
print(condition)
print(np.logical_not(condition))
```

Output:

```
[False  True  True]
[ True False False]
```

এটি filtering এ "not" condition প্রয়োগ করতে ব্যবহৃত হয়।

4. `np.logical_xor()`

একটি condition true হলে কিন্তু উভয়টি true না হলে true return করে (exclusive OR)।

```
x = np.array([True, False, True, False])
y = np.array([True, True, False, False])
print(np.logical_xor(x, y))
```

Output:

```
[False  True  True False]
```

Logical Reduction Functions

NumPy তে কিছু ফাংশন আছে যা পুরো array এর ওপর logical evaluation করে।

1. `np.any()`

যদি array এর অন্তত একটি element True হয়, তাহলে True return করে।

```
arr = np.array([0, 0, 1, 0])
print(np.any(arr))
```

Output:

```
True
```

2. `np.all()`

যদি array এর সব element True হয়, তাহলে True return করে।

```
arr = np.array([1, 1, 1])
print(np.all(arr))
```

Output:

```
True
```

এই দুইটি function data validation বা conditional checks এ প্রায়ই ব্যবহৃত হয়।

Combining Conditions

Multiple logical conditions একসাথে ব্যবহার করা যায় bitwise operators দিয়ে:

```
arr = np.array([5, 10, 15, 20])
result = (arr > 5) & (arr < 18)
print(result)
print(arr[result])
```

Output:

```
[False  True  True False]
[10 15]
```

এখানে & (AND), | (OR), এবং ~ (NOT) operators element-wise কাজ করে, এবং এগুলো logical_and() ইত্যাদির বিকল্প হিসেবে ব্যবহার করা যায়।

Logical Indexing in Real Scenarios

Filtering এবং data cleaning এ logical operations অনেক উপকারী।

```
scores = np.array([45, 67, 89, 32, 76])
passed = np.logical_and(scores >= 50, scores <= 100)
print("Passed Students:", scores[passed])
```

Output:

```
Passed Students: [67 89 76]
```

Summary Table

Function	Description	Example
np.logical_and()	দুটি condition true হলে True	`(arr > 10) & (arr < 20)`
np.logical_or()	যেকোনো একটি condition true হলে True	`(arr < 5)
np.logical_not()	Boolean invert করে	`~(arr > 15)`
np.logical_xor()	শুধুমাত্র একটিই true হলে True	`np.logical_xor(a, b)`
np.any()	অন্তত একটি True থাকলে True	`np.any(arr > 0)`
np.all()	সবগুলো True হলে True	`np.all(arr > 0)`

Real-world Example

Data analysis এ missing value বা condition ভিত্তিক filter তৈরিতে logical functions ব্যাপকভাবে ব্যবহৃত হয়।

```
import numpy as np

data = np.array([45, 50, np.nan, 60, 30])
valid = np.logical_and(~np.isnan(data), data >= 50)
filtered_data = data[valid]
print(filtered_data)
```

Output:

```
[50. 60.]
```

এভাবে আমরা invalid (NaN) বা low-score data বাদ দিয়ে clean dataset তৈরি করতে পারি।
এই পর্যায়ে তুমি NumPy এর সব গুরুত্বপূর্ণ logical functions এবং তাদের ব্যবহার সম্পর্কে জেনে গেছ — এখন তুমি সহজেই condition-based filtering এবং validation apply করতে পারবে বড় dataset এর ওপর।

ndarray Sorting

এই পর্যায়ে আমরা শিখব কিভাবে **NumPy ndarray** তে sorting করা যায়। Data analysis, ranking, এবং statistical computation এর ক্ষেত্রে data কে ascending বা descending ক্রমে সাজানো অত্যন্ত দরকারি। NumPy এর sorting functions এই কাজটি অত্যন্ত দ্রুত এবং memory-efficientভাবে করে।

Basic Sorting with `np.sort()`

`np.sort()` function array কে default ভাবে ascending order এ sort করে এবং একটি **new sorted copy** return করে।

```
import numpy as np

arr = np.array([5, 1, 8, 3, 2])
sorted_arr = np.sort(arr)
print(sorted_arr)
```

Output:

```
[1 2 3 5 8]
```

np.sort() মূল array পরিবর্তন করে না, বরং নতুন array return করে।

In-place Sorting with `.sort()`

যদি তুমি মূল array কেই sort করতে চাও, তাহলে `.sort()` method ব্যবহার করতে পারো — এটি **in-place** operation করে।

```
arr = np.array([7, 2, 9, 4])
arr.sort()
print(arr)
```

Output:

```
[2 4 7 9]
```

.sort() মূল array পরিবর্তন করে এবং নতুন copy তৈরি করে না।

Sorting Along an Axis

Multidimensional array এর ক্ষেত্রে তুমি নির্দিষ্ট **axis** অনুযায়ী sort করতে পারো।

```
arr = np.array([[3, 1, 2], [6, 5, 4]])
print(np.sort(arr, axis=1))  # row-wise sort
print(np.sort(arr, axis=0))  # column-wise sort
```

Output:

```
[[1 2 3]
[4 5 6]]
[[3 1 2]
[6 5 4]]
```

এখানে axis=1 মানে প্রতি row আলাদা ভাবে sort হয়েছে, আর axis=0 মানে column অনুযায়ী।

Sorting Order (Ascending & Descending)

NumPy সরাসরি descending order support করে না, তবে আমরা slicing বা negative sign দিয়ে সহজেই করতে পারি।

```
arr = np.array([10, 2, 7, 1])
desc_sorted = np.sort(arr)[::-1]
print(desc_sorted)
```

Output:

```
[10 7 2 1]
```

এখানে [::-1] slice array কে reverse করে descending order তৈরি করেছে।

Getting Sorted Indices — `np.argsort()`

`np.argsort()` function array এর sorted order অনুযায়ী index return করে। এটি useful যখন তুমি data rearrange করতে চাও।

```
arr = np.array([50, 10, 30, 20])
indices = np.argsort(arr)
print(indices)
print(arr[indices])
```

Output:

```
[1 3 2 0]


[10 20 30 50]
```

এটি sorted order অনুযায়ী element এর position দেয়।

Sorting Structured Arrays

Structured array বা record array এর ক্ষেত্রে আমরা নির্দিষ্ট field অনুযায়ী sort করতে পারি।

```
data = np.array([("Alice", 25), ("Bob", 30), ("Charlie", 20)],
dtype=[('name', 'U10'), ('age', 'i4')])
print(np.sort(data, order='age'))
```

Output:

```
[(Charlie, 20) (Alice, 25) (Bob, 30)]
```

এখানে data field age অনুযায়ী ascending order এ sort হয়েছে।

Partial Sorting — `np.partition()`

`np.partition()` function array কে এমনভাবে ভাগ করে যে নির্দিষ্ট index এর আগে থাকা elements ছোট, এবং পরেরগুলো বড়। এটি **QuickSelect algorithm** ব্যবহার করে।

```
arr = np.array([9, 4, 7, 1, 3, 8])
part = np.partition(arr, 2)
print(part)
```

Output:

```
[3 1 4 7 9 8]
```

এখানে প্রথম ৩টি element সবচেয়ে ছোট values (unsorted order এ) থাকে।

Real-world Example

Sorting data বিশ্লেষণের অন্যতম ধাপ, যেমন top scores বের করা:

```
scores = np.array([80, 95, 70, 60, 85])
rank = np.argsort(scores)[::-1]  # descending order
print("Rank Order:", rank)
print("Top 3 Scores:", scores[rank][:3])
```

Output:

```
Rank Order: [1 4 0 2 3]
Top 3 Scores: [95 85 80]
```

Summary Table

Function	Description	Example
np.sort()	New sorted copy return করে	`np.sort(arr)`
arr.sort()	In-place sort করে	`arr.sort()`
np.argsort()	Sorted order অনুযায়ী indices return করে	`np.argsort(arr)`
np.partition()	Partial sort করে (QuickSelect)	`np.partition(arr, k)`
np.sort(..., axis)	নির্দিষ্ট axis অনুযায়ী sort করে	`np.sort(arr, axis=1)`

এই পর্যায়ে তুমি ndarray sorting এর সমস্ত প্রয়োজনীয় function, তাদের কাজ, এবং practical ব্যবহার সম্পর্কে শিখে ফেলেছ — এখন তুমি সহজেই data কে ordered form এ প্রস্তুত করতে পারবে।

ndarray Searching

এই পর্যায়ে আমরা শিখব কিভাবে **NumPy ndarray** এর মধ্যে data **search** করা যায়। Data analysis এবং manipulation এর ক্ষেত্রে search operations যেমন নির্দিষ্ট value খোঁজা, শর্ত অনুযায়ী element বের করা বা index জানা — এগুলো খুবই গুরুত্বপূর্ণ।

Searching with `np.where()`

`np.where()` function conditional search এর জন্য ব্যবহৃত হয়। এটি এমন element এর index return করে যেগুলো নির্দিষ্ট condition পূরণ করে।

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])
indices = np.where(arr > 25)
print(indices)
print(arr[indices])
```

Output:

```
(array([2, 3, 4]),)
[30 40 50]
```

এখানে np.where() return করে একটি tuple of indices, যা condition true এমন সব positions দেখায়।

Conditional Replacement

`np.where()` দিয়ে শুধুমাত্র index নয়, condition অনুযায়ী value replace করাও সম্ভব।

```
arr = np.array([10, 20, 30, 40, 50])
new_arr = np.where(arr > 30, 1, 0)
print(new_arr)
```

Output:

```
[0 0 0 1 1]
```

condition অনুযায়ী True element গুলোকে 1 এবং False গুলোকে 0 তে replace করা হয়েছে।

Searching for Exact Value — `np.where()` + equality

```
arr = np.array([3, 7, 3, 2, 9, 3])
indices = np.where(arr == 3)
print(indices)
```

Output:

```
(array([0, 2, 5]),)
```

এখানে আমরা সব index পেয়েছি যেখানে element = 3।

`np.nonzero()` — Finding Non-zero Elements

`np.nonzero()` এমন সব index return করে যেখানে element শূন্য নয়।

```
arr = np.array([0, 2, 0, 5, 7, 0])
print(np.nonzero(arr))
```

Output:

```
(array([1, 3, 4]),)
```

এটি sparsity check বা mask তৈরি করতে খুব উপকারী।

`np.argmax()` এবং `np.argmin()`

এই দুইটি function array এর **maximum** এবং **minimum** value এর index return করে।

```
arr = np.array([12, 45, 7, 22, 50])
print(np.argmax(arr))  # index of max value
```

```
3print(np.argmin(arr))  # index of min value
```

Output:

```
1 4

2 2
```

np.argmax() data ranking বা sorting shortcut হিসেবেও ব্যবহৃত হয়।

Searching in Multidimensional Arrays

`np.where()` multi-dimensional array এর ক্ষেত্রেও কাজ করে।

```
1arr = np.array([[10, 15], [20, 25], [30, 35]])
2indices = np.where(arr > 20)
3print(indices)
4print(arr[indices])
```

Output:

```
1(array([1, 2, 2]), array([1, 0, 1]))
2[25 30 35]
```

এখানে দুটি array of indices return হয়েছে, যা row ও column position নির্দেশ করে।

`np.searchsorted()` — Insertion Index Finding

`np.searchsorted()` একটি sorted array এ নতুন value insert করার position return করে যাতে sorting order ঠিক থাকে।

```
1arr = np.array([10, 20, 30, 40, 50])
2pos = np.searchsorted(arr, 35)
3print(pos)
```

Output:

```
13
```

এটি insertion বা merge operation এর আগে index নির্ধারণ করতে ব্যবহৃত হয়।
তুমি চাইলে **left/right parameter** দিয়ে **insertion direction control** করতে পারো —

```
1arr = np.array([10, 20, 30, 30, 40])
2print(np.searchsorted(arr, 30, side='left'))
```

```
3print(np.searchsorted(arr, 30, side='right'))
```

Output:

```
1 2



2 4
```

এখানে side='left' মানে প্রথম 30 এর index, আর side='right' মানে শেষ 30 এর পরের index।

`np.extract()` — Conditional Extraction

`np.extract()` function condition অনুযায়ী array এর element বের করে আনে।

```
1arr = np.arange(10)
2condition = arr % 2 == 0
3result = np.extract(condition, arr)
4print(result)
```

Output:

```
1[0 2 4 6 8]
```

এটি arr[arr % 2 == 0] এর সমতুল্য, কিন্তু ফাংশন আকারে ব্যবহার সুবিধাজনক।

Real-world Example

Suppose, তোমার কাছে students এর score আছে এবং তুমি 60 এর উপরে score পেতে চাও:

```
1scores = np.array([55, 72, 48, 90, 67])
2passed_idx = np.where(scores >= 60)
3print("Indices:", passed_idx)
4print("Passed Scores:", scores[passed_idx])
```

Output:

```
1Indices: (array([1, 3, 4]),)
2Passed Scores: [72 90 67]
```

এভাবে তুমি সহজেই filtering, ranking, এবং condition-based searching apply করতে পারো।

Summary Table

Function	Description	Example
np.where()	Condition অনুযায়ী index বা value খোঁজে	`np.where(arr > 5)`
np.nonzero()	Non-zero elements এর index দেয়	`np.nonzero(arr)`
np.argmax() / np.argmin()	Max / Min value এর index দেয়	`np.argmax(arr)`
np.searchsorted()	Sorted array এ insertion index দেয়	`np.searchsorted(arr, x)`
np.extract()	Condition অনুযায়ী elements বের করে	`np.extract(cond, arr)`

এই পর্যায়ে তুমি ndarray searching এর সব প্রধান function, তাদের কাজ, এবং ব্যবহারিক প্রয়োগ বুঝে গেছ — এখন তুমি data থেকে condition-based extraction, ranking, এবং filtering apply করতে পারবে সহজেই।

ndarray Counting

এই পর্যায়ে আমরা শিখব কিভাবে **NumPy ndarray** এর মধ্যে data count বা frequency বের করা যায়। Data analysis, statistics, এবং feature engineering এর সময় element count জানা অনেক গুরুত্বপূর্ণ, যেমন কতগুলো element নির্দিষ্ট শর্ত পূরণ করছে বা প্রতিটি unique value কতবার এসেছে।

Counting with `np.count_nonzero()`

`np.count_nonzero()` function array এর মধ্যে কতগুলো element non-zero তা count করে।

```
import numpy as np

arr = np.array([0, 1, 2, 0, 3, 0, 4])
count = np.count_nonzero(arr)

print(count)
```

Output:

```
4
```

এটি boolean array এর ক্ষেত্রেও condition-based counting এর জন্য ব্যবহার করা যায়।

```
arr = np.array([10, 20, 30, 40, 50])
```

```
count = np.count_nonzero(arr > 25)
print(count)
```

Output:

```
3
```

এখানে আমরা গুনেছি কতগুলো element 25 এর বেশি।

Counting Specific Values

যদি তুমি নির্দিষ্ট value কতবার এসেছে জানতে চাও, তাহলে simple comparison ও `np.sum()` একত্রে ব্যবহার করতে পারো।

```
arr = np.array([1, 2, 3, 2, 4, 2, 5])
count = np.sum(arr == 2)
print(count)
```

Output:

```
3
```

এখানে arr == 2 একটি boolean array তৈরি করেছে, যার True গুলোর সংখ্যা sum করলে count পাওয়া গেছে।

Counting Unique Elements — `np.unique()`

`np.unique()` function array এর unique elements return করে, আর চাইলে তাদের frequency ও count করে দিতে পারে।

```
arr = np.array([1, 2, 1, 3, 2, 1])
unique, counts = np.unique(arr, return_counts=True)
print(unique)
print(counts)
```

Output:

```
[1 2 3]
[3 2 1]
```

এখানে আমরা পেয়েছি 1 তিনবার, 2 দুইবার এবং 3 একবার এসেছে।

Frequency Distribution (Dictionary Form)

`np.unique()` এর ফলাফল dictionary আকারেও তৈরি করা যায় —

```
arr = np.array(['apple', 'banana', 'apple', 'mango', 'banana'])
```

```
unique, counts = np.unique(arr, return_counts=True)
frequency = dict(zip(unique, counts))
print(frequency)
```

Output:

```
{'apple': 2, 'banana': 2, 'mango': 1}
```

এটি dataset এর category distribution বের করার জন্য ব্যবহৃত হয়।

Multidimensional Counting

`np.count_nonzero()` multidimensional array এর ক্ষেত্রেও কাজ করে, এবং তুমি চাইলে নির্দিষ্ট **axis** অনুযায়ী count করতে পারো।

```
arr = np.array([[0, 1, 2], [3, 0, 4]])
print(np.count_nonzero(arr, axis=0))  # column-wise
print(np.count_nonzero(arr, axis=1))  # row-wise
```

Output:

```
[2 1 2]


[2 2]
```

axis=0 মানে প্রতি column এ non-zero element কতটি আছে, আর axis=1 মানে প্রতি row এ।

Counting NaN Values

NaN (Not a Number) মানগুলোর count বের করতে আমরা `np.isnan()` ব্যবহার করি।

```
arr = np.array([10, np.nan, 5, np.nan, 8])
nan_count = np.count_nonzero(np.isnan(arr))
print(nan_count)
```

Output:

```
2
```

এটি missing value detection এর জন্য প্রায়ই ব্যবহৃত হয়।

Boolean Mask Counting

যদি কোনো condition অনুযায়ী True/False mask তৈরি করো, তাহলে তার count সহজেই বের করা যায়।

```
arr = np.array([12, 15, 18, 10, 25])
```

```
mask = arr > 15
print(mask)
print(np.sum(mask))
```

Output:

```
[False False  True False  True]
2
```

Mask এর True value গুলোই count হয়, যা conditional frequency জানার উপায়।

Real-world Example

Suppose, তোমার কাছে student grades আছে এবং তুমি জানতে চাও কতজন pass করেছে ও কতজন fail:

```
grades = np.array([55, 40, 78, 90, 30, 67])
passed = np.count_nonzero(grades >= 50)
failed = np.count_nonzero(grades < 50)
print(f"Passed: {passed}, Failed: {failed}")
```

Output:

```
Passed: 4, Failed: 2
```

এভাবে আমরা সহজেই dataset থেকে category-wise count পেতে পারি।

Summary Table

Function	Description	Example
np.count_nonzero()	Non-zero বা True element গুনে	`np.count_nonzero(arr)`
np.sum(condition)	Condition অনুযায়ী count করে	`np.sum(arr > 10)`
np.unique(..., return_counts=True)	Unique value ও তাদের frequency দেয়	`np.unique(arr, return_counts=True)`
np.isnan()	NaN মান সনাক্ত করে	`np.count_nonzero(np.isnan(arr))`

এই পর্যায়ে তুমি ndarray counting এর সকল গুরুত্বপূর্ণ পদ্ধতি শিখে ফেলেছ — এখন তুমি data validation, category frequency, এবং condition-based count বের করতে NumPy দক্ষতার সাথে ব্যবহার করতে পারবে।

Statistical Functions

এই পর্যায়ে আমরা শিখব **NumPy Statistical Functions** সম্পর্কে — যেগুলো data এর ওপর বিভিন্ন statistical measure যেমন mean, median, standard deviation, variance ইত্যাদি calculate করতে ব্যবহৃত হয়। এই functions data analysis, machine learning এবং scientific computation এ অত্যন্ত গুরুত্বপূর্ণ।

Mean — `np.mean()`

`np.mean()` function array এর সব element এর গড় (average) value return করে।

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])
mean_value = np.mean(arr)
print(mean_value)
```

Output:

```
30.0
```

এটি (sum of elements) / (number of elements) হিসেব করে।
তুমি চাইলে নির্দিষ্ট **axis** অনুযায়ী **mean** বের করতে পারো —

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(np.mean(arr, axis=0))  # column-wise
print(np.mean(arr, axis=1))  # row-wise
```

Output:

```
[2.5 3.5 4.5]
[2. 5.]
```

Median — `np.median()`

`np.median()` function array এর median (middle value) return করে।

```
arr = np.array([5, 2, 8, 1, 9])
median_value = np.median(arr)
print(median_value)
```

Output:

```
5.0
```

যদি total element সংখ্যা even হয়, তবে median হয় দুইটি middle element এর average।

Standard Deviation — `np.std()`

`np.std()` array এর standard deviation return করে — অর্থাৎ data কতটা ছড়ানো তা নির্দেশ করে।

```
arr = np.array([10, 12, 23, 23, 16, 23, 21, 16])
std_value = np.std(arr)
print(std_value)
```

Output:

```
4.898979485566356
```

Standard deviation বেশি মানে data বেশি spread out, কম মানে data কাছাকাছি।

Variance — `np.var()`

`np.var()` function variance হিসেব করে, যা standard deviation এর square।

```
arr = np.array([10, 20, 30, 40, 50])
var_value = np.var(arr)
print(var_value)
```

Output:

```
200.0
```

Variance data variability পরিমাপ করে।

Minimum and Maximum — `np.min()` এবং `np.max()`

এই দুটি function array এর সর্বনিম্ন ও সর্বোচ্চ মান return করে।

```
arr = np.array([4, 9, 1, 7, 3])
print(np.min(arr))
print(np.max(arr))
```

Output:

```
1
9
```

Data normalization এর সময় এগুলো প্রায়ই ব্যবহার করা হয়।

Percentile — `np.percentile()`

`np.percentile()` function dataset এর percentile মান return করে — অর্থাৎ, কত শতাংশ data সেই মানের নিচে রয়েছে।

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
print(np.percentile(arr, 50))  # median
print(np.percentile(arr, 90))  # 90th percentile
```

Output:

```
5.0
8.2
```

90th percentile মানে 90% data 8.2 এর নিচে আছে।

Weighted Average — `np.average()`

`np.average()` function weighted average হিসেব করে, যেখানে প্রতিটি element এর ওজন আলাদা হতে পারে।

```
arr = np.array([10, 20, 30])
weights = np.array([1, 2, 3])
weighted_avg = np.average(arr, weights=weights)
print(weighted_avg)
```

Output:

```
23.333333333333332
```

এখানে বেশি weight যুক্ত element গুলোর প্রভাব বেশি।

Correlation Coefficient — `np.corrcoef()`

`np.corrcoef()` দুটি variable এর মধ্যে correlation (সম্পর্ক) পরিমাপ করে।

```
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])
print(np.corrcoef(x, y))
```

Output:

```
[[1. 1.]
[1. 1.]]
```

Correlation coefficient +1 মানে perfect positive relation, -1 মানে perfect negative relation।

Real-world Example

ধরা যাক, তোমার কাছে student marks dataset আছে — তুমি চাইছো average, highest, lowest এবং deviation বের করতে।

```
marks = np.array([78, 85, 92, 67, 88, 76])
print("Mean:", np.mean(marks))
print("Median:", np.median(marks))
print("Std Dev:", np.std(marks))
print("Max:", np.max(marks))
print("Min:", np.min(marks))
```

Output:

```
Mean: 81.0
Median: 81.5
Std Dev: 8.4
Max: 92
Min: 67
```

এভাবে statistical functions ব্যবহার করে dataset এর পুরো summary তৈরি করা যায়।

Summary Table

Function	Description	Example
np.mean()	Average value হিসেব করে	`np.mean(arr)`
np.median()	Middle value return করে	`np.median(arr)`
np.std()	Standard deviation দেয়	`np.std(arr)`
np.var()	Variance দেয়	`np.var(arr)`
np.min(), np.max()	সর্বনিম্ন ও সর্বোচ্চ মান	`np.min(arr)` , `np.max(arr)`
np.percentile()	Percentile মান দেয়	`np.percentile(arr, 90)`
np.average()	Weighted average দেয়	`np.average(arr, weights=w)`
np.corrcoef()	Correlation coefficient দেয়	`np.corrcoef(x, y)`

এই পর্যায়ে তুমি NumPy এর সব গুরুত্বপূর্ণ statistical functions এবং তাদের প্রয়োগ সম্পর্কে জেনে গেছ — এখন তুমি data summarize, analyze, এবং interpret করতে পারবে দ্রুত ও দক্ষতার সাথে।

Linear Algebra with NumPy

এই পর্যায়ে আমরা শিখব কিভাবে **NumPy** ব্যবহার করে Linear Algebra সম্পর্কিত operations perform করতে হয়। NumPy তে linear algebra এর জন্য একটি শক্তিশালী submodule রয়েছে — `numpy.linalg`। এটি matrix operations, determinant, inverse, eigenvalues, এবং solving linear equations এর মতো কাজের জন্য ব্যবহৃত হয়।

Importing the Linear Algebra Module

NumPy তে linear algebra functions ব্যবহার করতে আলাদা করে কিছু import করতে হয় না — এগুলো `np.linalg` submodule এর মধ্যে থাকে।

```
import numpy as np
```

Matrix Representation

Matrix কে সাধারণত 2-D ndarray হিসেবে তৈরি করা হয়।

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A)
print(B)
```

Output:

```
[[1 2]
[3 4]]
[[5 6]
[7 8]]
```

এখানে A এবং B দুটি 2×2 matrix।

Matrix Multiplication — `np.dot()` / `@`

`np.dot()` function বা `@` operator ব্যবহার করে matrix multiplication করা যায়।

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[2, 0], [1, 2]])
C = np.dot(A, B)
```

```
4print(C)
```

Output:

```
1[[ 4  4]
2[10  8]]
```

অথবা —

```
1C = A @ B
2print(C)
```

দুটি matrix multiply করার সময় প্রথম matrix এর column সংখ্যা এবং দ্বিতীয় matrix এর row সংখ্যা সমান হতে হবে।

Matrix Transpose — `np.transpose()` বা `.T`

Transpose মানে row কে column এ এবং column কে row এ রূপান্তর করা।

```
1A = np.array([[1, 2, 3], [4, 5, 6]])
2print(A.T)
```

Output:

```
1[[1 4]
2[2 5]
3[3 6]]
```

এটি mathematical notation এ A^T হিসেবে প্রকাশ করা হয়।

Determinant — `np.linalg.det()`

`np.linalg.det()` function কোনো square matrix এর determinant হিসাব করে।

```
1A = np.array([[1, 2], [3, 4]])
2print(np.linalg.det(A))
```

Output:

```
1-2.0000000000000004
```

Determinant শূন্য হলে matrix singular হয় এবং inverse বের করা সম্ভব হয় না।

Inverse of a Matrix — `np.linalg.inv()`

`np.linalg.inv()` function দিয়ে matrix এর inverse বের করা যায়।

```
1A = np.array([[1, 2], [3, 4]])
```

```
invA = np.linalg.inv(A)

print(invA)
```

Output:

```
[[-2.   1. ]
 [ 1.5 -0.5]]
```

Inverse matrix এমন একটি matrix যা মূল matrix এর সাথে multiply করলে identity matrix দেয়।

Identity Matrix — `np.eye()`

`np.eye()` function identity matrix তৈরি করতে ব্যবহৃত হয়।

```
I = np.eye(3)

print(I)
```

Output:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Identity matrix এর diagonal এ 1 থাকে এবং বাকি সব 0।

Solving Linear Equations — `np.linalg.solve()`

`np.linalg.solve()` function ব্যবহার করে আমরা linear equation system solve করতে পারি।

উদাহরণ:

ধরা যাক,

```
2x + y = 8

x - y = 2
```

তাহলে —

```
A = np.array([[2, 1], [1, -1]])
B = np.array([8, 2])
X = np.linalg.solve(A, B)

print(X)
```

Output:

```
[3.33333333 1.33333333]
```

অর্থাৎ x = 3.33 এবং y = 1.33।

Eigenvalues and Eigenvectors — `np.linalg.eig()`

`np.linalg.eig()` function matrix এর eigenvalues এবং eigenvectors return করে।

```
A = np.array([[4, -2], [1, 1]])
values, vectors = np.linalg.eig(A)
print("Eigenvalues:", values)
print("Eigenvectors:\\n", vectors)
```

Output:

```
Eigenvalues: [3. 2.]
Eigenvectors:
[[0.894 0.707]
[0.447 0.707]]
```

Eigenvalues matrix এর transformation behavior বোঝায়, যা ML ও PCA তে ব্যবহৃত হয়।

Matrix Norm — `np.linalg.norm()`

`np.linalg.norm()` function matrix এর magnitude বা length নির্ধারণ করে।

```
A = np.array([[3, 4]])
print(np.linalg.norm(A))
```

Output:

```
5.0
```

এটি Euclidean distance বা vector length হিসেবেও ব্যবহৃত হয়।

Rank of a Matrix — `np.linalg.matrix_rank()`

Matrix rank মানে matrix এর independent row বা column এর সংখ্যা।

```
A = np.array([[1, 2], [2, 4]])
print(np.linalg.matrix_rank(A))
```

Output:

```
1
```

Rank 1 মানে দ্বিতীয় row প্রথমটির multiple, তাই linearly dependent।

Real-world Example

Machine Learning এ linear algebra operations খুব সাধারণ — যেমন regression equation solve করা, transformation apply করা বা PCA তে covariance matrix এর eigenvalue বের করা।

```
1A = np.array([[2, 1], [1, 3]])
2B = np.array([8, 18])
3X = np.linalg.solve(A, B)
4print(X)
```

Output:

```
1[3. 5.]
```

এখানে সমীকরণ system 2x + y = 8 এবং x + 3y = 18 এর সমাধান x=3, y=5।

Summary Table

Function	Description	Example
np.dot() / @	Matrix multiplication করে	`A @ B`
np.transpose() / .T	Matrix transpose করে	`A.T`
np.linalg.det()	Determinant হিসেব করে	`np.linalg.det(A)`
np.linalg.inv()	Inverse matrix দেয়	`np.linalg.inv(A)`
np.eye()	Identity matrix তৈরি করে	`np.eye(n)`
np.linalg.solve()	Linear equations solve করে	`np.linalg.solve(A, B)`
np.linalg.eig()	Eigenvalues ও eigenvectors দেয়	`np.linalg.eig(A)`
np.linalg.norm()	Matrix norm (magnitude) দেয়	`np.linalg.norm(A)`
np.linalg.matrix_rank()	Matrix rank হিসেব করে	`np.linalg.matrix_rank(A)`

এই পর্যায়ে তুমি NumPy এর Linear Algebra সম্পর্কিত সমস্ত গুরুত্বপূর্ণ function, তাদের ব্যবহার ও বাস্তব প্রয়োগ সম্পর্কে জেনে গেছ — এখন তুমি matrix operations, system solving, এবং eigenvalue computation দক্ষতার সাথে করতে পারবে।